

Problema B

Considere uma tabela Hash que utiliza endereçamento aberto, mas com um detalhe adicional: **as chaves podem ser removidas**, e essas remoções devem ser tratadas usando **marcação lógica**.

- Uma posição removida não é considerada vazia; ela é marcada com um valor especial **-1**.
- Ao inserir uma nova chave, **posições marcadas como -1 podem ser reutilizadas**.
- A busca deve continuar sondando até:
 - encontrar a chave procurada, ou
 - encontrar uma posição vazia (valor 0).

A função Hash é a mesma utilizada na semana anterior:

$$h(x) = x \% M, \text{ onde } x \text{ é a chave e } M \text{ é o número de endereços-base.}$$

Para tratar colisões, utiliza-se a função de endereçamento aberto:

$$h(x) = (x+i)\%M, \text{ para } i=1,2,\dots$$

Operações

Cada caso de teste contém uma sequência de operações:

- **Inserir a chave x:** I x
- **Remover a chave x** (marcar endereço com -1, se encontrado na tabela): R x
- **Buscar a chave x:** B x

É garantido que não haverão operações com os valores 0 e -1.

Entrada

A primeira linha contém um inteiro N, o número de casos de teste. A primeira linha de cada caso de teste é composta por dois inteiros: M, o tamanho da tabela Hash e O, o número de operações a serem realizadas. Depois disso, seguem O linhas, cada uma contendo uma operação no formato especificado acima.

Saída

Para cada caso de teste, imprima a posição em que foi encontrado o elemento após a operação de busca. Caso não seja encontrado nenhum elemento, imprima -1.

Após cada caso de teste, imprima uma quebra de linha adicional.

Exemplo de entrada	Exemplo de saída
1 10 7 I 15 I 25 I 35 R 25 B 35 I 55 B 25	7 -1

Exemplo de entrada	Exemplo de saída
2 10 12 I 15 I 25 I 35 B 25 R 25 B 25 I 45 I 55 R 35 I 75 B 35 B 55 7 10 I 14 I 21 I 28 B 21 R 14 I 35 B 14 I 42 R 21 B 28	6 -1 -1 8 1 -1 2

Você pode utilizar a seguinte estrutura para desenvolver o seu programa:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct TabelaHash {  
    int *tabela;  
    int tamanho;  
} TabelaHash;
```

```
int funcaoHash(int chave, int tamanho) {  
    return abs(chave) % tamanho;  
}
```

```
TabelaHash *criaTabelaHash(int tamanho){  
    TabelaHash *th = (TabelaHash*) malloc (sizeof(TabelaHash));  
  
    // comando calloc já aloca um vetor preenchendo todos os valores com  
    0  
    th->tabela = (int*) calloc (tamanho, sizeof(int));  
    th->tamanho = tamanho;  
    return th;  
}
```

```
int main(){  
    int n;  
    scanf("%d\n", &n);  
  
    for(int i = 0; i < n; ++i){  
        int tamanho, o;  
        scanf("%d %d\n", &tamanho, &o);  
        TabelaHash *tabela_hash = criaTabelaHash(tamanho);  
  
        for(int j = 0; j < o; ++j){  
            char operacao;  
            int chave;  
            scanf("%c %d\n", &operacao, &chave);  
  
            /* COMPLETAR */  
        }  
    }  
  
    return 0;  
}
```